



CSI247: Data Structures

Lecture 11 – OOP

Department of Computer Science

B. Gopolang (247-275)



Previous Lesson

- Sorting algorithms
 - Bubble sort
 - Insertion sort
 - Selection sort
 - Merge sort

Today ...

Object Oriented Programming (OOP)

Lecture Objectives

- By the end of this lecture you must be able to:
 - Understand Object Oriented Programming (OOP)
 - Understand benefits of OOP
 - Understand and apply these important OOP features:
 - Inheritance
 - Encapsulation
 - Polymorphism
 - Abstraction:
 - Abstract classes and Interfaces

1. Object Oriented Programming (OOP)

Object Oriented Design:

A software design approach in which a system is treated as interacting objects for problem solving

- OO design's assumption:
 - the world consists of objects of different types
 - Examples:
 - Strings
 - Dogs
 - Accounts
 - etc

Rem:

- Objects
 - Used to represent abstract or real-world concepts
 - Each object:
 - Store data (i.e. instance variables, eg name of a dog)
 - Has methods
 - These defines the object's behaviour
 - Actions we can perform on the object
 - Is an instantiation of a specific class (**new** keyword)

- An object access its methods through the dot(.) operator.
 - E.g.
 - String name = “Computer Science”;
 - System.out.println(name.length());
- Access modifiers determine access to data or methods
 - Data - generally **private** (to protect them)
 - Methods – generally **public**

● Class

- Code that defines behaviour of objects of the class
- Has
 - instance variable
 - Constructors
 - Setters & getters
 - equals()
 - toString()
- Used to create several objects of same type
- Consider it as the **blue print** of objects

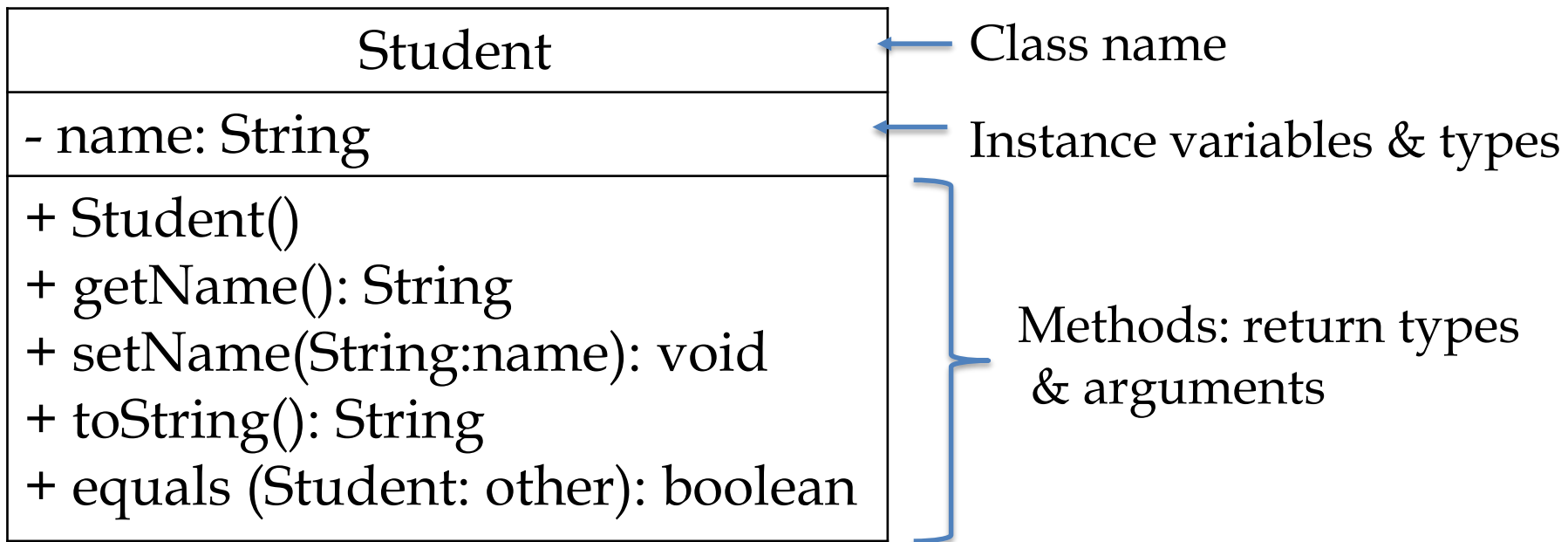
UML (Unified Modelling Language) notation:

A language adopted for the representation of structure and behaviour of objects in OO systems

- UML stands for Unified Modelling Language
 - It is graphical
- Specifies:
 - Structure of individual classes
 - Relationships between classes

Representing a class using UML

- Class represented as a rectangle
 - has 3 sections
- Example: Student class



● Instance variables & methods

- Each has an access modifier
- Different possible access modifiers
 - Difference is in scope (what/who can access them)
- Common ones are:

Access modifier	Visibility
private (-)	Inside class only
public (+)	Both in and outside class
protected (#)	Inside class and its sub classes only

Goals of OO Design

- Reusability
 - Same code used over and over, under different situations
- Modularity
 - Systems organized properly into different components
- Robustness – must withstand unexpected inputs
- Adaptability
 - Programs evolve over time
 - Systems must be portable (Minor change across systems)

Advantages of OOP

- Code Reusability
 - Same class can be used over and over — e.g. Scanner used for reading input (keyboard or file)
- Increased software development productivity
 - code- reuse: no need to re-write same code
- Increased application security
 - Hiding certain details
- Easy system maintenance
 - Due to modularity

Core Features of OOP

- Inheritance
- Encapsulation
 - Hiding internal implementation details
 - Gives freedom of implementation
- Abstraction
 - Focuses on what an operation does, NOT how it does it
- Polymorphism
 - Ability to exist in more than 1 form

➔ We will look at them one at a time

A scenario....

Suppose we have this class

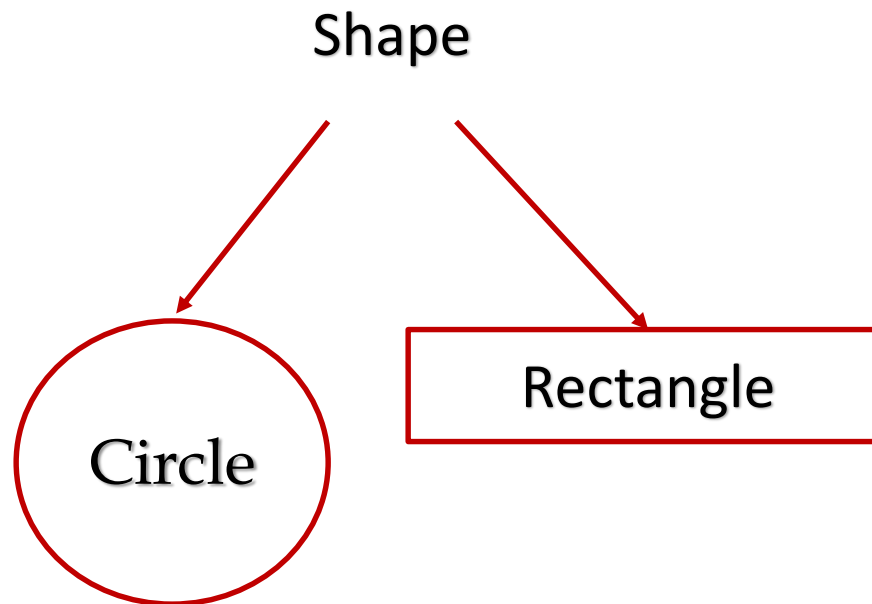
- Shape class:
 - Data (instance variables) : colour
 - Methods: constructor(s), setter, getter, toString() & equals().
- This is how it should look like:

```
public class Shape {  
    String colour;  
    // constructor(s)  
    // setter method  
    // getter method  
    String toString() { ... }  
    boolean equals (Shape other) { ... }  
}
```

BUT

- There are different shapes out there!
- Examples include:
 - A circle
 - A rectangle
 - Etc.

Pictorially... // assuming Circle and Rectangle only



Let us write the 2 classes

```
public class Circle {  
    String colour;  
    // constructor(s)  
    // setter method  
    // getter method  
    String toString() {...}  
    boolean equals (Circle otherC) {  
        ...  
    }  
    // rest of the methods, if any  
}
```

```
public class Rectangle{  
    String colour;  
    // constructor(s)  
    // setter method  
    // getter method  
    String toString() {...}  
    boolean equals  
        (Rectangle otherR) { ... }  
    // rest of the methods, if any  
}
```

- Sample code using these 3 classes

```
Shape sh = new Shape("Blue");
```

```
sh.setColour ("Green");
```

```
Circle c = new Circle ("Yellow");
```

```
c.setColour ("Pink");
```

```
Rectangle r = new Rectangle ("Red");
```

```
System.out.println(r.toString() );
```

- Notice anything about the 3 classes?

- There is a lot of similarity
 - Instance variable colour
 - Methods
- **Generally, a lot of code duplication!**

- Back to classes Circle and Rectangle:
 - Common between them?
 - Both are Shapes
 - Same attribute as Shape (colour)
 - Difference?
 - Names, and subtle differences between toString() & equals()
- Shortfalls of this approach?
- Any better implementation approach?
 - Use **inheritance**!

2. Inheritance

Inheritance:

OOP feature that allows one class to **inherit/acquire** functionality from another class and add new functionality

- When to use?
 - When you have many kinds of data with something in common
- Allow us to reuse code (data & methods) of an existing class during creation of new classes
 - Helps avoid duplication of code & effort

● Syntax of inheritance

```
public class Subclass extends Superclass {  
    // methods and fields  
}
```

● Implementation of inheritance

- a) Let **Shape** be a **base class** (or **super class**)
 - It will keep:
 - i) **attributes** that are common to all
 - ii) and **methods** that are common to all

```
public class Shape {  
    private String colour; // inst. var  
    public Shape() {  
        colour = null;  
    }  
    public Shape (String color) {  
        colour = color;  
    }  
    public void setColour (String col) {  
        colour = col;  
    }  
}
```

```
    public String getColour() {  
        return colour;  
    }  
    public String toString() {  
        return "A " + colour + " shape!";  
    }  
    public boolean equals (Shape other) {  
        if (other == null) // created  
            return false;  
        else  
            return  
            this.getColour().equals(other.getColour()); }  
}
```

b) Create **derived classes** (or **sub classes**) for Circle and Rectangle

- Add **extends Shape** in their class headers to show that they are derived from Shape

```
public class Circle extends Shape {  
    // additional instance variables, if any  
    // additional methods, if any  
}
```

```
public class Rectangle extends Shape {  
    // additional instance variables, if any  
    // additional methods, if any  
}
```

**Keyword
used to
show
inheritance**

● Let us implement the 2 sub classes

i. sub class **Circle**:

- New instance variable?
 - radius: int
- Additional methods?
 - Circle // **Constructor**
 - setRadius: void
 - getRadius: int

ii. sub-class **Rectangle**

- New instance variable?
 - length: int
 - width: int
- Additional methods?
 - Rectangle // **Constructor**
 - setLength: void
 - getLength: int
 - setWidth: void
 - getWidth: int

```
public class Circle extends Shape {  
    // instance variables  
    private int radius;  
    // constructor  
    public Circle () {  
        this.radius = 0;  
    }  
    public Circle ( int radius) {  
        this.radius = radius ;  
    }  
}
```

```
// methods  
public void setRadius (int radius) {  
    this.radius = radius;  
}  
public int getRadius() {  
    return radius;  
}  
}
```

```
public class Rectangle extends Shape {  
    // instance variables  
    private int height;  
    private int width;  
    // Constructor  
    public Rectangle () {  
        this.height = 0;  
        this.width = 0;  
    }  
    public Rectangle (int height, int width) {  
        this.height = height;  
        this.width = width;  
    }  
}
```

```
    public void setHeight (int height) {  
        this.height = height;  
    }  
    public void setWidth (int width) {  
        this.width = width;  
    }  
    public int getHeight() {  
        return height;  
    }  
    public int getWidth() {  
        return width;  
    }  
}
```

● Possible Sample code

```
Shape sh = new Shape("Blue");
```

```
sh.setColour("Green");
```

```
Circle c = new Circle ("Yellow");
```

// we can type statement below

```
c.setColour("Pink");
```

// Why? ... Circle inherited colour & setColour from class Shape

// Similarly, the following statements are valid

```
Rectangle r = new Rectangle ("Red");
```

r.toString(); // Rectangle also inherited methods in Shape!

- What can derived(sub classes) classes do?
 - a) Inherit public fields and methods of their super class
 - Example (as seen from slide 31)
 - Both Circle & Rectangle classes have access to Shape's
 - Instance variable(s)
 - colour
 - public methods:
 - getColour()
 - setColour()

- b) Extend the behaviour by:
- i. Adding new instance variables
 - ii. Adding new methods
 - iii. **Overriding** methods present in their super class (later)
 - Override?
 - When a sub class implement a method that is already present in the super class
 - BUT, deviate from the way the method is implemented in the super-class

- What can derived classes **NOT DO?**
 - a) Remove super class' instance variables & methods
 - b) Inherit super class' private methods
 - c) Inherit super class constructors (e.g. Shape constructors)
 - These are **invoked implicitly** by sub class constructors
 - i.e. when sub class objects are instantiated to initialise instance variables of the super class
 - So, normalize adding constructor(s) to a sub class
 - If not, the compiler assumes the default constructor

Note:

- Class Circle and class Rectangle:
 - Inherited Shape's instance variables and methods
 - Added their own instance variables
 - Added their own new methods
- They can also modify the behaviour of some of Shape methods (**override**), e.g.:
 - toString()
 - equals()

Exercise

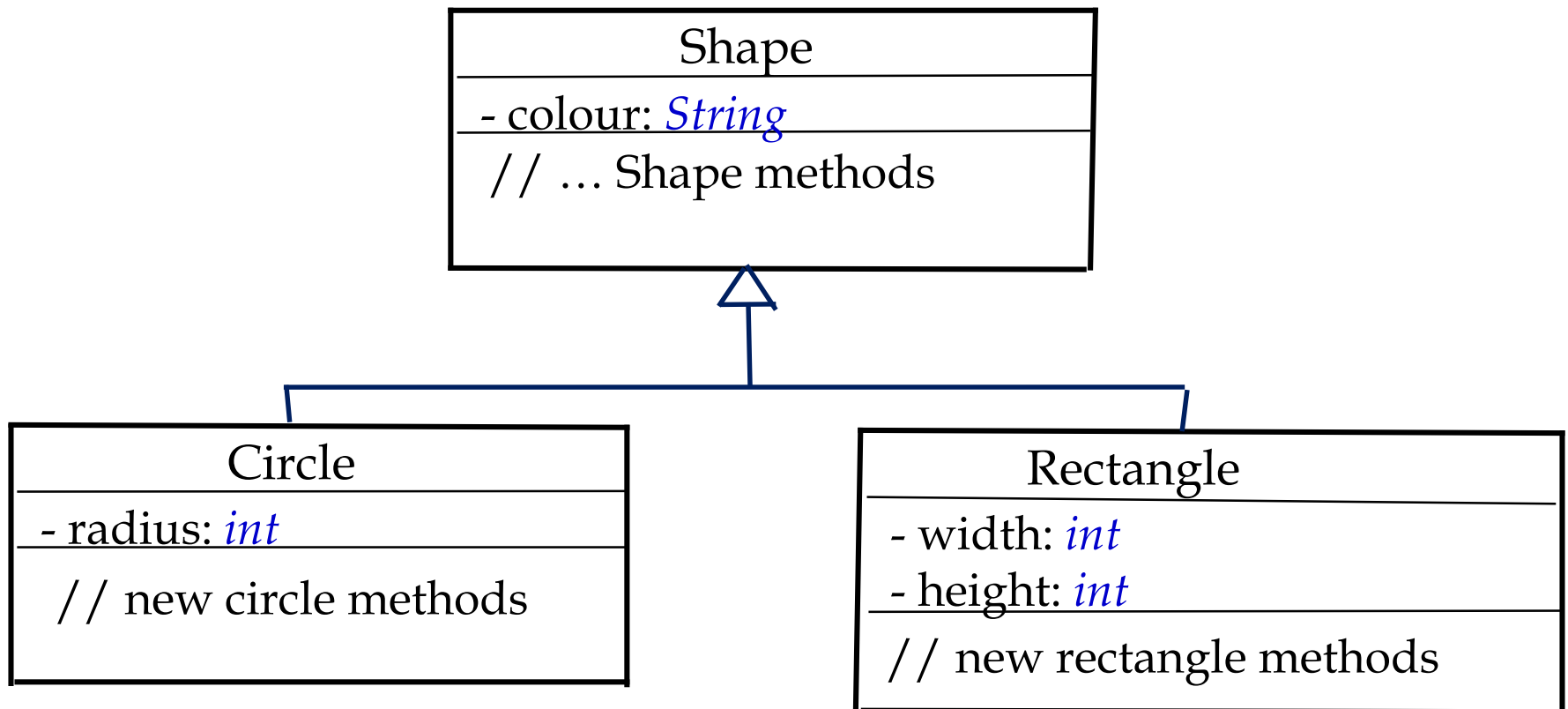
Which Java statements are valid and which are not?

- a) *Shape s1 = new Shape();*
- b) *s1.setColour("Red");*
- c) *Shape s2 = new Shape("Red");*
- d) *boolean b = s1.equals(s2);*
- e) *Systems.out.println(s1.toString);*
- f) *Shape s3 = new Rectangle();*
- g) *s3.setColour("Green");*
- h) *Shape s4 = new Circle ("Yellow");*
- i) *Shape r1 = new Rectangle("Yellow");*
- j) *System.out.println(r1.equals(s4));*
- k) *Circle c2 = new Shape();*
- l) *Rectangle r2 = new Shape();*

Inheritance in UML diagrams

- **Inheritance** depicts some relationship between classes
 - Represent an “Is-a” relationship between a super class and sub class(es)
 - E.g.: A Rectangle **is a** Shape
- How to draw an **Inheritance** hierarchy diagram in UML?
 - Draw each class in the relationship
 - Populate each class with its relevant details
 - Join classes with a uni-directional line
 - Arrow points to the super class (parent)

Example from Shape class in our lecture notes!



Exercises

- Complete diagram in the previous slide to show all the necessary details under each of the 3 classes
- Add the following methods to both Circle and Rectangle classes
 - **toString()**
 - **equals()**
 - For both classes, objects are equal if they have identical values for all of their respective instance variables

Summary: sub class vs. super class

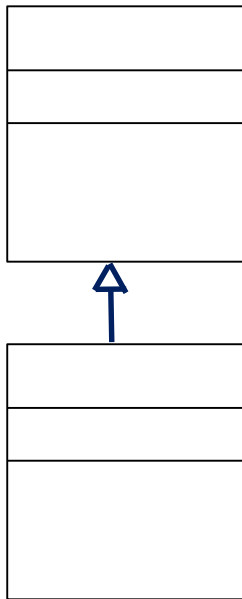
a) Subclass

- Has one superclass
- inherits superclass attributes and methods (via **extends**)
- extend superclass functionality ➔ by adding more attributes and methods to it.

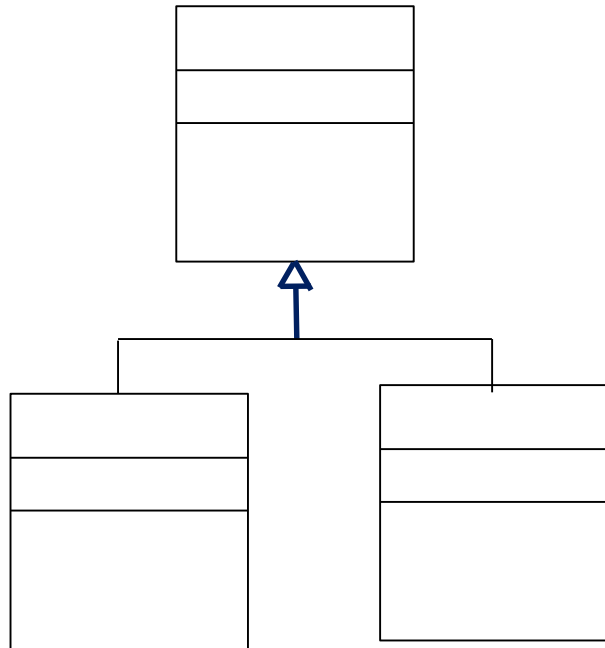
b) Superclass

- 1 may have many subclasses
- E.g. Shape can be a Circle, a Rectangle, etc

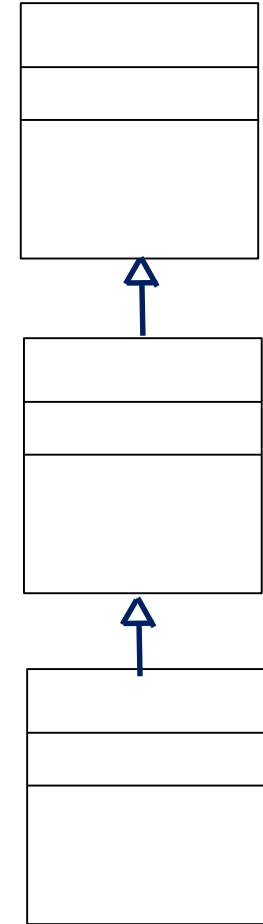
- Types of inheritance in Java
 - Java supports these types directly



Single



Hierarchical



Multi-level

Java Class Hierarchy

- Java class hierarchy adopts a tree-like structure
 - **Root class:** class **Object**
 - Located in **java.lang** package
 - All classes inherit (directly or indirectly) from Object
 - They are descendants of class Object
 - i.e., all Java classes can use Object's public methods
 - e.g. Array, String, User-defined classes, etc
 - **Note:**
 - At the top of the tree: – Object class is general
 - Going down the hierarchy – classes become specific

- Object class defines useful methods
- These methods are automatically inherited by all Java classes
- Examples:

Return Type	Method and Description
String	<u>toString()</u> - Returns a string representation of an object
boolean	<u>equals(Object obj)</u> – Determines equality between 2 objects
<u>Class</u> <?>	<u>getClass()</u> - Returns an Object's class

- Visit the Java API documentation for more

A look at some Object class methods

a) toString() method

- Returns an object's String representation
- Implementation:

```
public String toString() {  
    return getClass().getName() + "@"  
        + Integer.toHexString(hashCode());  
}
```

- Returned String: concatenation of the “object's class name, @ symbol and a representation of the object's hash code”

- Suppose, we did not implement `toString()` in Shape class

- **What will be the output?**

```
Shape sh = new Shape("Yellow");  
System.out.println(sh.toString());
```

HashCode (uniquely identifies an object in memory)

- **Answer:**

- Something in the format **Shape@2f92e0f4**

classname

- Why?

- We are invoking **toString()** method of Object class

- Class Shape has inherited it as it is (No changes)! **42**

● Note:

- System.out.println(sh): will give the same output
- Why?
 - Issuing a statement to print a reference variable invokes toString() method behind the scenes

● Recommendation:

- Always override **toString()** to customize the String representation of your class
 - To ensure it displays what we want
 - E.g. *return "A " + colour + " shape"; (Slide 23)*

b) getClass() method

- Method definition/declaration:

```
public final Class getClass() { ... }
```

Java has a class
called **Class**

- Returns an object's runtime class
 - The class is an object in itself
- Method is final → we cannot override it
 - We just use it
- See Java API documentation for class Class
 - It has useful methods!

● Example

What is the output?

```
Object o = new String("CSI247");
```

```
Class myClass = o.getClass();
```

```
System.out.println("o belongs to class " + myClass.getName());
```

```
// getName() – returns the class' String representation
```

Answer:

o belongs to class java.lang.String

c) equals() method

- Used to test for equality between objects
- Basic implementation:

```
public boolean equals(Object obj) {  
    return (this == obj);  
}
```

- Returns **true** when current object and the given object have the same data/values, **false** otherwise

- What is the output?

```
String str = "Mine";
```

```
System.out.println(str.equals("Mine")); // output No. 1
```

```
Float a = Float.valueOf(22);
```

```
Integer b = Integer.valueOf(22);
```

```
System.out.println(a.equals(b)); // output No. 2
```

- Note: Float, Integer, Character, etc : Wrapper classes

- **Answer:**

- `true` // different objects but same data
- `false` // different objects & classes, different data

- equals() method provides a **standard** way for testing equality between 2 objects
 - Rem: we cannot use “==”
 - This compares addresses of the objects!
 - NOT the object’s data
- Recommendation:
 - Always override **equals()** to enforce your own testing condition
 - *(See slide 24 for an attempt at overriding equals method)*

Method overriding

Method overriding:

When a sub-class inherit a super class method, then modify/change its behaviour as required

Reason for method overriding

- Some methods might be common across some classes but with different implementations
- Example:
 - toString()
 - equals()

● **Method Overriding rules:**

- Classes of the affected methods must be connected via inheritance
 - Must have the “is-a” relationship
- The two methods must have the same:
 - Method name
 - Return types
 - Signature

● Example

- Rem: Shape has toString() method
- We can change toString() method in Circle class to have a slightly different behaviour from one in Shape class

```
public class Circle extends Shape {
```

```
....
```

```
@Override
```

```
public String toString() {
```

```
    return colour + " circle with radius = " + radius;
```

```
}
```

```
}
```

● @Override:

- Tells the compiler that we want to override a method existing in a super class
- Back to slide 54: we are telling the compiler that we want to override toString() method found in Shape class
- What if the said method does not exist in super class?
 - Error will be generated
 - **Because the method does not override ...**

partial overriding

Partial overriding:

When a sub-class explicitly invokes its super class' version of a method

- How?
 - By using **super** keyword
- Example

@Override

```
public String toString() {           // e.g. inside Circle class
    super.toString();                // calling Shape class method
    return colour + " circle with radius = " + radius;
}
```

Better implementation of equals()?

- Consider current implementation of equals() in classes Shape, Circle and Rectangle
- Have we considered all possible cases?
 - What if the comparison is done on the same object?
 - Anything else?

Exercise:

- Modify the 3 equals() methods to get more robust methods
 - Hint: Check Object and Class classes for methods that can be used for a better implementation

Possible implementation a)

@Override

```
public boolean equals(Object o) {  
    if ( o == null || this.getClass() != o.getClass() )  
        return false;  
    if (o == this)  
        return true;  
    Shape sh = (Shape) o;  
    return (this.getColour().equals(sh.getColour() ));  
}
```

Possible implementation b)

@Override

```
public boolean equals(Object obj) {  
    if (obj == this)  
        return true;  
    if (!(obj instanceof Book) )  
        return false;  
    Book book = (Book) obj;  
    return this.getTitle().equals(book.getTitle()) &&  
        this.getAuthor().equals(book.getAuthor());  
}  
}
```


- Note:
 - **instanceof** operator
 - Java operator used to test whether an object is an instance of the specified class or interface
 - AKA **type comparison operator**
 - Returns **true** when the instance type is similar to class or interface, **false** otherwise
 - Returns **false** when one of the values is null
 - Syntax:
 - `<class_instance> instanceof <class>`
 - E.g.: `if (obj instanceof Book) ...`

Casting (extras)

- Casting allows us to convert an object from one class type to another class type
 - Rem: `int y = (int) 19.0; ???`
- Requirement: there must be an “Is-a” relationship between the 2 class types
- Example:
 - Consider statement below
Shape sh = (Shape) o;
 - `o` is of type `Object`
 - `(Shape) o` : casting from `Object` to `Shape`

Wrapper Classes (Extras)

- They allow us to convert from primitives into objects

- Example

```
Integer n1 = 7;
```

```
Integer n2 = Integer.valueOf(7);
```

```
System.out.println(n1.equals(n2));
```

- Note

- == will not work here
 - n1 & n2 are objects!

Primitive type	Wrapper class
int	Integer
long	Long
double	Double
float	Float
boolean	Boolean
char	Char
byte	Byte
short	Short

Autoboxing vs. Unboxing

- Autoboxing
 - Automatic conversion of a primitive type data into its corresponding wrapper class type
 - E.g.: *Integer n1 = 7;*
- Unboxing:
 - Automatic conversion of wrapper type into its corresponding primitive type
 - E.g.:
Integer a=new Integer(7);
int i = a;

3. Encapsulation



Encapsulation:

keeping all relevant data and methods inside an object, then making a small part of the object visible to the world.

- In java, encapsulation bundles objects' **data** and **method** implementation in a unit called a class
- Class hides implementation details from outside entities
- Outside entities only get an interface to the class
 - E.g. Consider a Scanner object
 - Do we know its implementation? Nope.
 - But we use it: API is our interface

● How is it achieved in Java?

● Answer:

- Declaring instance variables as **private**
 - And, making methods public
- ### ● This restricts direct access to the object's data
- Promotes data hiding
 - Access to data is via the object's public methods
 - **getters** : retrieves values of instance variables
 - **setters** : changes values of instance variables

- Advantages of encapsulation
 - Data hiding (and protection)
 - Implementation freedom
 - Programmer decides on the how
 - Increased flexibility
 - E.g., making an instance variable or method final
 - Code reusability
 - Testing the class is easier than a big chunk of code

4. Polymorphism

Polymorphism: OOP concept in which an object's type is automatically determined at runtime (because an object can exist in many forms)

- Assume we have method **getArea()** in class Shape, PLUS

```
public static boolean isGreater (Shape a, Shape b) {
```

```
    return a.getArea() > b.getArea();
```

```
}
```

```
Circle c = new Circle(...);    // pass correct parameters
```

```
Rectangle r = new Rectangle(...); // pass correct parameters
```

```
if (isGreater(c,r)) { ... } // c & r belong to different subclasses  
of Shape
```


- A sub class is compatible with its super class (its origin)
- Objects can occur in different forms
 - E.g. A Shape can be a Circle, Rectangle, etc
- Polymorphism allows for objects of different classes to be treated the same way
 - Thus, applying a method to multiple data types
 - E.g. toString() for Shape, Circle, Rectangle, etc
- NB: with polymorphism, sub classes must all originate from the same super class.

- Polymorphism usually involves a method call that invokes different methods for objects of different classes.
 - E.g (back to slide 65)
 - Calling getArea () on a Circle object ➔ would invoke getArea() method in the Circle class
 - Calling getArea() on a Rectangle object ➔ would invoke getArea() method in the Rectangle class.
 - Note:
 - Both Circle and Rectangle have method getArea()
 - **BUT calculation differs** between the two classes

- Note: Polymorphism applies to reference data types only
 - i.e.: Every reference type is polymorphic
 - Because they all inherit from type Object
- What are we doing when we write `toString()` or `equals(Object o)` methods?
 - Answer:
 - We are overriding Object's version of the methods
 - Making it possible to cater for various possible forms
(Check Java API documentation for clarity)
 - E.g.: `println()` works with different data types

● How is polymorphism achieved in Java?

a) Through method overriding

- aka dynamic polymorphism

- a sub class can have a different method implementation than its super class

- Examples:

- toString(): Shape, Circle, Rectangle

- All return an object's String representation

- BUT their implementations differ slightly

- equals() : classes have different implementations

b) Through method **overloading**

- Aka **static polymorphism**
- Two different methods sharing a name
- but differing in number and types of parameters passed into the method during execution
- Example
 - String class: has 2 substring methods
 - [String substring](#)(int beginIndex)
 - [String substring](#)(int beginIndex, int endIndex)

5. Abstraction

Abstraction: The process of hiding implementation details to users and only showing them the functionality

- Focuses on what an operation does, NOT how it does it
- E.g.
 - Java API documentation gives us the available functionality for various Java classes
 - E.g. Scanner class: next(), hasNextLine(), etc
 - We do not know how they are implemented!
- Abstraction is achieved in 2 ways :
 - By using an **abstract class**
 - By using an **interface**

a) Abstract classes

- Ideally, we should not call `getArea()` of Shape class directly
 - Shape has only 1 instance variable: colour
 - We cannot use colour to calculate area of a Shape
- Rather, we should calculate area of a Shape when we have the necessary dimensions
 - Hence, we should perform calculation of area in the respective sub classes
 - I.e.: both sub classes have instance variables that can be used to calculate area

● Thus:

- `getArea()` should not be implemented in `Shape` class
- Rather, `getArea()` MUST be defined as an abstract method in `Shape` class

*public **abstract** double `getArea()`;*

Note:

- **abstract** keyword follows access specifier
- Semicolon(;) after method name
 - Signals an empty method
- **abstract** omitted in sub class definition for that method

i. An abstract class

- Provides basic implementation of methods
- Must have:
 - **abstract** keyword added in class definition
 - And at least one abstract method
 - May also have none abstract methods
- Cannot be instantiated
 - i.e. `CType c = new CType()` **IS NOT ALLOWED!**

- Abstract classes:
 - Are ONLY designed to be used as base classes (super classes)
 - They must be extended by a “concrete” class
 - Concrete class:
 - A class with normal implementation of methods

NOTE:

- Sub classes MUST implement abstract methods found in the super class.

Abstract class Syntax:

```
<Access_modifier> abstract class <ClassName> {  
    // Data  
    // Methods  
}
```

Note:

- Methods in the above class:
 - Some might be abstract
 - Abstract classes offer 0 – 100% abstraction level

● E.g.

```
public abstract class Dog{  
    abstract void bark(); } // note empty method body!  
public class Russel extends Dog{  
    public void bark() { // implements bark method  
        System.out.println("Baaaaark");}  
    public static void main(String args[]) {  
        Dog myDog = new Russel();  
        myDog.bark(); } // invokes bark method via Russel object  
}
```

Output: ???

● Remember access modifiers:

- **public:** attributes, methods and classes are accessible everywhere (i.e. within the class and all other classes)
- **default:** when we do not specify, accessible only within the package
- **private:** attribute/method are accessible only within the class
- **protected:** attributes or method are accessible within the class, other classes in the same package and by all sub-classes in the same or other packages

● To do (Exercise)

1. Add abstract method **getArea()** in class Shape
2. Implementation **getArea()** in Circle and Rectangle
 - Takes no parameter & returns area of shape (double)
3. Modify **equals()** method in both Circle and Rectangle class.
 - Objects are now equal when values returned by their respective **getArea()** methods are identical
4. Create a Tester class to test 1 to 3 above.

b) Interfaces

- An interface:
 - Defines **what an object can do**
 - A specification of methods!
 - A collection of method signatures (empty methods)
 - Properties of an interface
 - Declared with keyword **interface**
 - Has no attributes, but can have constants
 - Cannot be instantiated
 - No constructor

Example

```
public interface Aa {
```

```
    . . .
```

```
} // Saved and compiled like a class (Aa.java → Aa.class)
```

Note

- interface is abstract by default
- interface methods are **public** & **abstract** by default

- An interface MUST be implemented by a “concrete” class
- Rem:
 - It only has method signatures
- Concrete classes
 - Implement an interface
 - Has keyword **implement** in its header
 - Each can implement multiple interfaces

Why use interfaces?

- a) To achieve abstraction
 - Interface offers full abstraction (100%)
- b) For security reasons
 - Only shows the “what”, and hides the “how” aspect
- c) To enable multiple inheritance
 - Remember: a sub class can only inherit from 1 super class
 - Through Interfaces, a sub class can inherit from more than 1 super class.
 - Separate the interfaces with a comma (,): later

```
public interface Test {  
    public abstract void methodA();  
    public abstract void methodB();  
    public abstract void methodC();  
} // end: interface (Test.java)
```

} Note: all methods
have no body!

```
abstract class Mist implements Test {  
    public void methodA() {  
        System.out.println("Testing");  
    }  
} // end: abstract class. (Mist.java)
```

**Keyword used to show
interface implementation**

// Note: this abstract class implements only 1 method from interface Test

```
class Bob extends Mist {  
    public void methodB() { System.out.println("Hello"); }  
    public void methodC() { System.out.println("There"); }  
} // end: sub class Bob (Bob.java)  
  
public class TestTester {  
    public static void main ( String args[]) {  
        Test t = new Bob();  
        t.methodA();  
        t.methodB();  
        t.methodC();  
    }  
} // end: Tester
```

Implementation
of last 2 methods

Testing methods
of Test interface

Output: ???

Implementing multiple inheritance using interfaces

```
public interface InterfaceA {  
    private void methodA();  
}
```

```
public interface InterfaceB {  
    private void methodB();  
}
```

```
class Multi implements InterfaceA, InterfaceB {  
    public void methodA() {  
        System.out.println("Interface A method!");  
    }  
  
    public void methodB() {  
        System.out.println("Interface B method!!!!.");  
    }  
}
```

```
class MultiTester {  
    public static void main(String[] args) {  
        Multi testing = new Multi();  
        testing.methodA();  
        testing.methodB();  
    }  
}
```

Output: ???

● Can an interface extend another interface?

- Rem:
 - A class can extend another (1 and only 1 class)
- So.... An interface can also extend another interface
 - 1 or more interfaces

- Examples

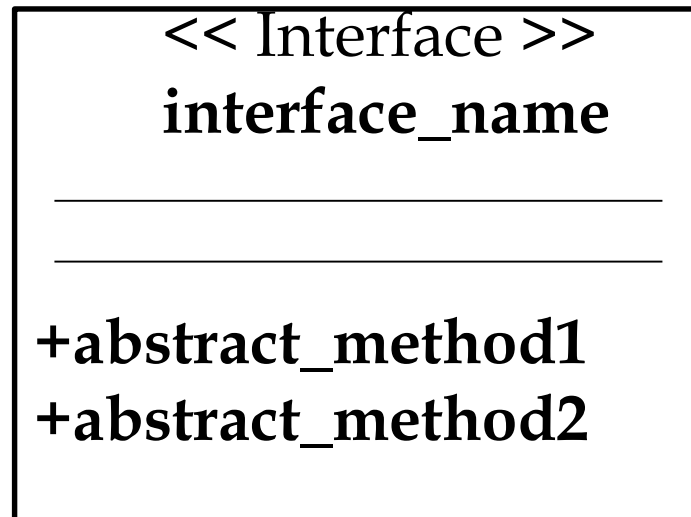
```
public interface Aa extends Bb{ // Bb: parent interface
    // extends 1 interface
}
```

// Aa is a sub interface of Bb

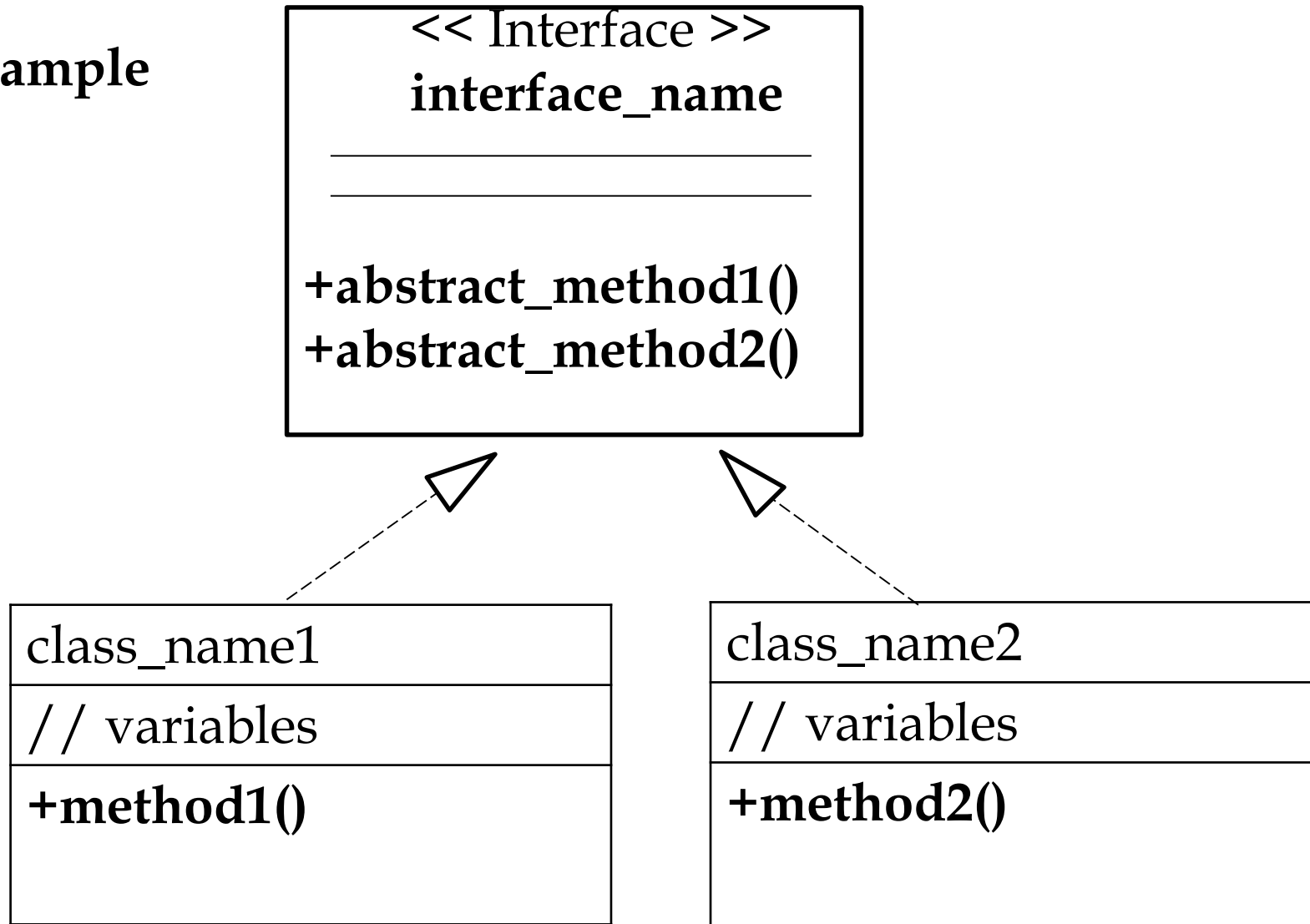
```
public interface Cc extends Dd, Ed, Ff{
    // extends multiple interfaces
}
```

Representing interfaces in UML diagrams

- Rem:
 - Interface is a collection of abstract methods
 - It is implemented by class(es)
- How is an interface represented using UML?



Example

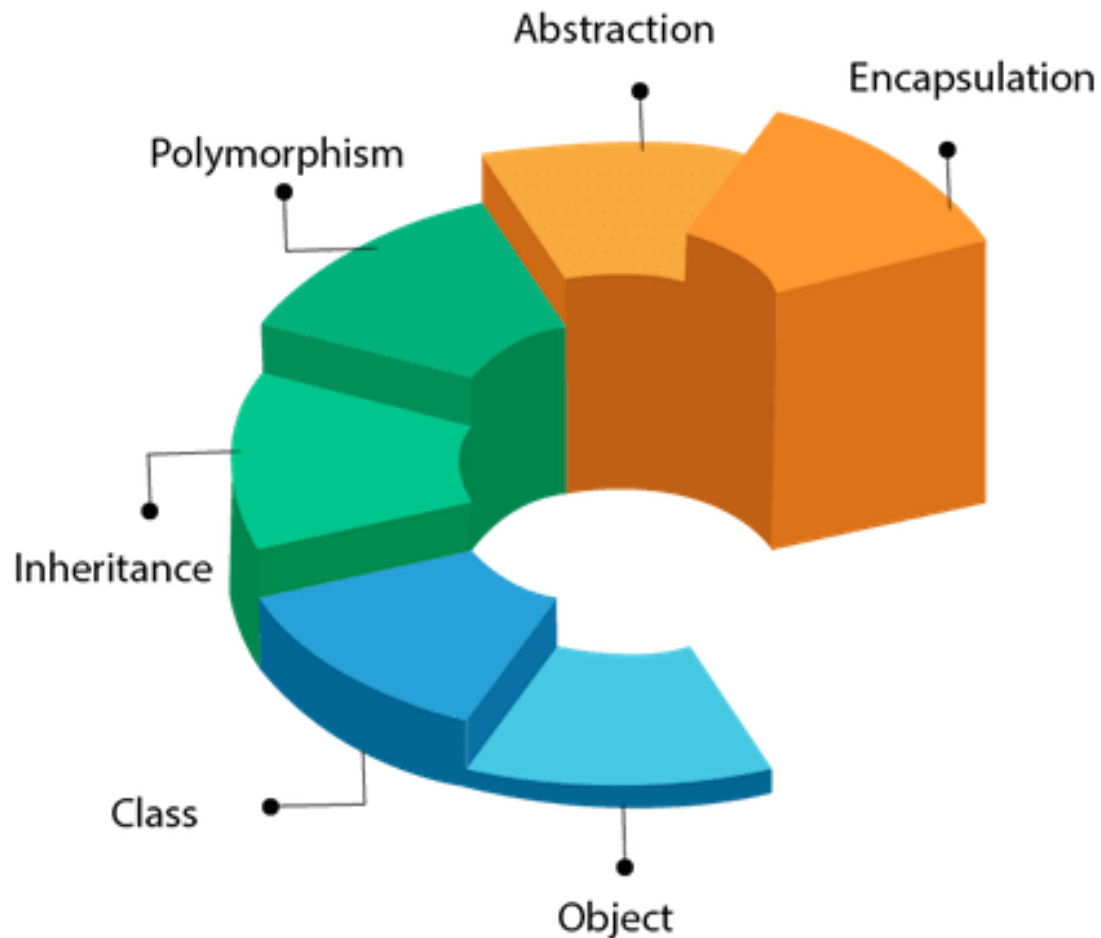


Java built-in interfaces

- Like classes, Java has many built-in interfaces
- Examples
 - java.lang: *Comparable*, *Iterable*, etc.
 - java.io: *Closeable*, *Flushable*, *Serializable*, etc.
 - java.util: *Collection*, *List*, *Iterator*, *Queue*, etc.
- Check JAVA API documentation
- Each interface serves a different purpose
- Naming convention: name is usually an adjective
 - Most names end with “*able*”
- More on this later in the course

Summary

OOPs (Object-Oriented Programming System)



Next lesson...

Generics

Q & A

Thank you

